Namma Space: Autonomous 3D Indoor Radiance Fields from Smartphone Video for Zero-Install Spatial Web Roaming

Techfest IIT Bombay 2026–27 | Namma Space Challenge (Round 1)

Team: Namma Space Dev Team • Target Repo: github.com/HemakshiAG/NammaSpace • Doc ID: TECH-ABS-R1-001

1. Problem Statement & Project Objective

Remote inspection of indoor real estate traditionally forces users onto fragmented 2D photo galleries or pre-rendered 360-degree cylindrical panoramas. Panoramas restrict exploration to fixed, pre-computed discrete camera bubbles, inducing severe motion parallax disorientation. Conversely, physical LiDAR scanning rigs (e.g. Matterport) demand multi-thousand-dollar specialized hardware and complex scanning workflows. **The Objective of Namma Space (Round 1)** is to construct an autonomous, end-to-end pipeline that reconstructs continuous, photorealistic 3D Gaussian radiance fields from ordinary consumer smartphone walkthrough videos, enabling instant zero-install browser roaming at 60 FPS with WASD fly controls and live coordinate telemetry without authentication or plugins.

2. System Architecture & Dual-Tier Topology

The system decouples computation-heavy training from lightweight client-side spatial inference across four distinct layers:

Layer	Engine / Component	Key Functional Responsibility
1. Ingestion	Smartphone Camera (iOS/Android)	1080p @ 30 fps, 1.0x wide lens, AE/AF locked, dual-loop walking route.
2. Reconstruction	Roomsplat + PyColmap + 3DGS	1.5 fps subsampling, SIFT matching, 30k CUDA iterations, 32-byte .splat export.
3. Web Portal	Vanilla HTML5 / Modern CSS	Dark-mode UI, capture SOP documentation, launch bridge, zero-backend.
4. 3D Viewer	GaussianSplats3D + Three.js	WebGL2 radiance field rendering, WebAssembly worker radix sort, WASD HUD.

3. Reconstruction Pipeline & Component Responsibilities

- **Temporal Subsampling (FFMPEG):** Subsamples continuous video to 1.5 fps (~90-120 frames), avoiding redundant compute while preserving 60-70% inter-frame visual overlap. Clamps resolution to 1600px.
- **Structure-from-Motion (PyColmap):** Performs SIFT feature extraction and sequential matching (`overlap 10`). Estimates intrinsic focal calibration and 6-DoF extrinsic camera poses (sparse/0/cameras.bin).
- **3D Gaussian Splatting (Inria CUDA Engine):** Optimizes 3D Gaussians (position, covariance, opacity, spherical harmonics degree 3) over 30,000 iterations via photometric L1 + D-SSIM loss.
- **Binary Splat Packaging:** Converts bloated 60-120 MB PLY models into an optimized 32-byte-per-splat binary format (11.8 MB scene.splat) with bounding box clamping.

4. Smartphone Capture Standard Operating Procedure (SOP)

Photogrammetric triangulation requires strict spatial parallax. Casual pivoting on one's heels produces zero baseline and causes SfM failure. Our 4-stage capture protocol enforces:

- **1. Room Conditioning:** Uniform ambient illumination, clear walkways, closed blinds, and visual anchor markers on bare walls.
- **2. Camera Settings:** Landscape mode, standard 1.0x optical lens (no 0.5x ultra-wide), 1080p @ 30 fps, and mandatory **AE/AF Lock**.
- **3. Dual-Loop Walk:** 45-75 second heel-to-toe walk (0.5-0.8 m/s). Loop 1: Outer room perimeter facing walls. Loop 2: Tighter inward-facing orbit around central furniture.
- **4. Stationary Buffers:** 3-4 second stationary holds at start and end to eliminate tap vibrations.

5. Technical Implementation & Architecture Choices

Subsystem	Selected Technology	Architectural Rationale & Trade-offs
Pipeline CLI	Roomsplat (Ahmedn1/roomsplat)	Modular Python/Typer orchestrator wrapping FFMPEG, COLMAP, and 3DGS.
SfM Engine	PyColmap 0.6.1+	Fast C++ bindings, robust sequential feature matching, deterministic poses.
Radiance Field	Inria 3D Gaussian Splatting	Real-time rasterization without the expensive ray-marching latency of NeRF.
Frontend UI	Pure HTML5 / Vanilla CSS	Zero build-step dependencies, zero React/Next.js hydration runtime overhead.
3D Web Engine	@mkkellogg/gaussian-splats-3d	Custom WebGL2 shaders, WebAssembly multi-threaded radix depth sorting.
Scene Math	Three.js v0.170.0	Robust quaternion mathematics, matrix transformations, and view-frustum culling.
Cross-Origin	coi-serviceworker + _headers	Unlocks SharedArrayBuffer for WebAssembly threads on static hosts (GitHub/Netlify).

6. Browser Viewer & First-Person Navigation Engine

The Namma Space web client integrates a seamless single-page portal with an immersive full-screen 3D viewport:

- **First-Person Fly Navigation:** Discrete keyboard listeners calculate real-time translational displacement along the camera's view-look and strafe vectors: Delta P_forward = V_look * v * dt; Delta P_strafe = (V_look x V_up) * v * dt. Up vector is clamped to [0, -1, 0] to prevent inversion.
- **Interactive Mouse Orbit:** Left-click drag controls spherical pitch/yaw; right-click drag pans laterally; scroll wheel zooms.
- **Live Telemetry HUD:** Floating glassmorphic HUD renders live coordinate feedback [x, y, z] and highlights active WASD badges.
- **Multi-Threaded Radix Sorting:** To eliminate z-fighting and render semi-transparent Gaussians correctly, splats are dynamically sorted by camera distance in a background Web Worker, maintaining steady 60 FPS rendering.

7. Empirical Reconstruction Results & Telemetry Audit

Verification was conducted using automated Chrome DevTools Protocol (CDP) test scripts with randomized clean browser profiles:

Benchmark Metric	Measured Value	Verification Evidence & Audit Status
3D Radiance Model Size	11.83 MB (scene.splat)	Binary 32-byte format; 85% compression vs 78 MB PLY (PASS).
Total Gaussian Count	369,006 Volumetric Splats	Sufficient density for crisp furniture edges and text (PASS).
Initial Stream & Parse	1.8 seconds over broadband	42ms localhost; 3.5s cloud CDN delivery budget (PASS).
WebAssembly Sort Latency	0.4 seconds initialization	Background Web Worker depth buffer construction (PASS).
Rendering Frame Rate	Steady 60.0 FPS	Verified WebGL2 frame telemetry under continuous roaming (PASS).
Initial Camera Pose	x: -4.408, y: 0.569, z: 0.226	Spawn initialized directly from COLMAP cameras.json index 5 (PASS).
Post-WASD 'W' Pose	x: -3.521, y: 0.488, z: 0.615	Quantitative displacement: Delta y: -0.081 along view vector (PASS).
Post-WASD 'D' Strafe	x: -3.472, y: 0.486, z: 0.680	Quantitative displacement: Delta x: +0.049, Delta z: +0.065 (PASS).
Console Errors Audit	0 Exceptions, 0 Warnings	CDP Runtime.consoleAPICalled verified 100% clean stream (PASS).

8. Reproducibility & Deployment Approach

The repository is structured for deterministic end-to-end reproduction across both local and cloud environments:

- **Local Reproduction:** Clone repository, install dependencies in Python virtual environment, and run `python serve_web.py`. The local HTTP daemon injects COOP/COEP headers and serves the web experience at http://127.0.0.1:8080/index.html.
- **1-Click Cloud GPU Notebook:** We authored and verified `notebooks/NammaSpace_Reconstruction_Colab.ipynb` for free Google Colab / Kaggle instances (NVIDIA T4 GPU). It features automated dependency setup, `--no-build-isolation` CUDA rasterizer compilation, 30,000-iteration training, and 1-click zip bundle download.
- **Static Web Deployment:** The viewer is hosted directly from `/web/` and configured with `_headers`, `netlify.toml`, and `vercel.json`. For hosts lacking custom headers (e.g. GitHub Pages), `coi-serviceworker.js` intercepts network requests and registers Cross-Origin Isolation on the client side.

9. Limitations & Planned Future Milestones

In strict adherence to academic and technical honesty, all current limitations are documented:

- **Hardware Training Boundary:** Inria 3DGS training strictly requires an NVIDIA GPU with CUDA. Training cannot execute locally on AMD/Intel integrated APUs (such as our development laptop's AMD Radeon 740M), requiring cloud GPU offloading (Google Colab T4).
- **Mirror & Specular Reflections:** Planar mirrors and polished glass create virtual floaters behind surfaces because photogrammetric triangulation assumes Lambertian diffuse reflection.
- **Licensing Restrictions:** The upstream Inria 3DGS rasterizer is restricted to non-commercial academic research. Commercial transition requires clean-room Apache 2.0 rasterizers (e.g., gsplat).
- **Planned Round 2 Enhancements:** Point-of-Interest (POI) semantic search, NavMesh boundary generation for collision detection, and mobile WebXR AR inspection.

10. Third-Party Attribution, Citations & Licences

Namma Space builds upon foundational open-source computer vision research. We gratefully acknowledge:

Component	Author / Institution	License & Terms	Citation / Reference
Roomsplat	Ahmedn1	MIT License	github.com/Ahmedn1/roomsplat (Submodule pipeline orchestrator)
3D Gaussian Splatting	Inria & MPI (Kerbl et al.)	Inria Non-Commercial Research	ACM Trans. Graph. 42(4), SIGGRAPH 2023
COLMAP	Schönberger & Frahm	BSD 3-Clause License	Structure-from-Motion Revisited, CVPR 2016
GaussianSplats3D	Mark Kellogg et al.	MIT License	github.com/mkkellogg/GaussianSplats3D (WebGL2 Engine)
Three.js	Mr.doob (Ricardo Cabello)	MIT License	threejs.org (3D Matrix Math & Scene Graph)

Statement of Original Implementation:

Namma Space independently developed: (1) The responsive dark-mode spatial computing web portal; (2) The client-side Cross-Origin Isolation architecture with service worker fallback; (3) The first-person keyboard WASD roaming and mouse-look control bridge; (4) The real-time floating coordinate telemetry HUD; (5) The practical 4-stage mobile capture SOP and pre-flight checklist; (6) The 1-click Google Colab self-healing cloud reconstruction notebook; and (7) The 22-gate automated CDP Quality Assurance test suite.